RC low-pass: two standard values for a 1 kHz cutoff

The first and simplest genetic-algorithm experiment — value genome, milestone M5, CLAUDE.md §4

kronspice simulation report · 2026-11-07 · population 40, seed 1

© 2026 fdamador@comcast.net

Summary

A first-order RC low-pass filter is driven by a 1 V AC source. A genetic algorithm chooses the resistor from the E24 series and the capacitor from the E12 series so that the −3 dB frequency of V(out)/V(in) lands at 1 kHz. Everything is in a config file: the netlist is plain SPICE, and *cmd/evolve* reads the genes, the objective and the GA settings from *config.json*.

- **Result:** R1 = 160 Ω, C1 = 1 µF, giving $f_c = 1/(2\pi RC)$ = 994.7 Hz, an error of 0.53 %.
- **Found at once:** seed 1 stops after generation 0 (40 evaluations), because the random initial population already contains a design under the 0.6 % target.
- **The limit is the value grid, not the search:** 0.53 % is the closest any E24 × E12 pair gets to 1 kHz (the other such pair, 1.6 kΩ with 100 nF, has the same product).
- **The written design reproduces:** *evo.TestRCLowpassExample* re-parses *best.cir* and measures the same cutoff from it, within 10 Hz of the target.

Background

CLAUDE.md §4 makes the value genome the starting encoding: fixed topology, genes that are component values on a log scale, optionally snapped to E-series. Milestone M5 is done when this RC low-pass example hits its target cutoff within 1 %. It is a standard engineering exercise; no hypothesis is involved. The other two config-driven M5 examples are *negative_resistance_gain* and *rectifier_filter*.

Circuit, genes and objective

Template netlist (*circuit.cir*), with the starting values:

```
RC low-pass, values to be evolved
V1 in 0 AC 1
R1 in out 10k
C1 out 0 10n
.ac dec 100 10 100k
.end
```

Gene	Range	Series
R1	100 Ω … 1 MΩ	E24
C1	100 pF … 10 µF	E12

Objective	Target	Error
upper −3 dB edge of V(out)	1000 Hz	|f − 1000| / 1000

GA settings: population 40, up to 40 generations, seed 1, a 2 s deadline per evaluation, and *target_fitness* 0.006, which stops the run as soon as a design is within 0.6 % of the target.

© 2026 fdamador@comcast.net

Result

Part	Value
R1	160 Ω
C1	1 µF
Cutoff $f_c = 1/(2\pi RC)$	994.7 Hz (error 0.53 %)
Generations / evaluations	stopped after generation 0 / 40

doc/M5.md records the same outcome: R1 = 1.6 kΩ / 160 Ω with C1 = 100 nF / 1 µF (E24 × E12), f_c = 994.7 Hz (0.53 %, the closest any E24 × E12 pair gets).

Validation

- *evo.TestRCLowpassExample* runs this config exactly as *cmd/evolve* does, checks the cutoff is within 10 Hz, re-parses the written *best.cir* and measures the same cutoff from it.
- Exported template (*export/rc_lowpass_values_circuit_results.md*, 3 elements, 2 nodes, AC at 1000 Hz): Kron's diakoptic solve tears it automatically into 2 pieces joined by 1 link (R1, carrying 5.32018×10^{-5} A at phase 57.86°). The torn and full MNA solves differ by 5.6×10^{-17} of the largest node voltage, and the power taken by the R, L, C branches is the same in both (Roth 1959 §5). V(out) of the template is 0.846733 V at phase −32.14°.
- Run outputs go to *runs/<timestamp>-rc_lowpass_values/*: a copy of the config, *stats.csv* (best, mean and worst fitness per generation), *best/genNNNN.cir*, *best.cir* and *checkpoint.json*. *best.cir* opens in ngspice, LTspice and QSpice as it is.

Limits

- Only E24 resistors and E12 capacitors are allowed, so 0.53 % is the best achievable error; a finer series would be needed to get closer.
- The objective is the cutoff alone; component tolerance, source and load impedance are not modelled.

Reproducing the results

```
go run ./cmd/evolve -config examples/rc_lowpass_values/config.json   # about 1 s
go test ./evo -run TestRCLowpassExample                              # runs this config end to end
KRONSPICE_EXPORT=1 go test ./internal/export                          # writes export/
```

See also doc/M5.md (the value-genome engine, run directories, penalties).

© 2026 fdamador@comcast.net